

ACTIVIDAD FILÓSOFOS MULTIHILOS

1. Introducción

El Problema de la Cena de los Filósofos es un experimento mental clásico en la informática teórica y la programación concurrente. En el problema se visualiza la necesidad de coordinar el acceso a recursos escasos y compartidos por parte de procesos o hilos concurrentes.

El escenario lo componen N filósofos sentados en una mesa circular, con un tenedor ubicado entre cada par de ellos. Para que un filósofo pueda ejecutar su actividad de comer, debe adquirir dos tenedores simultáneamente, el de su izquierda y el de su derecha. Todo esto conlleva diseñar una actividad de manera que garantice tres puntos claves para el ejercicio:

- Nunca dos filósofos deben usar el mismo tenedor a la vez
- El sistema nunca debe llegar a un estado donde todos los hilos esperen indefinidamente la liberación de un recurso.
- Ningún filósofo debe esperar permanentemente, garantizando que todos coman a menudo.

Objetivo de la actividad

El principal objetivo de esta actividad es implementar una solución al problema implementando los mecanismos de sincronización de Java.

- Implementar cada filósofo como un hilo
- Diseñar una clase monitor que gestione el acceso a los tenedores mediante las palabras clave *synchronized* y sincronización condicional (*wait()* / *notifyAll()*).
- Demostrar mediante depuración y pruebas unitarias la correcta prevención del *deadlock* y el mantenimiento de la invariante de seguridad.

2. Diseño de la solución

Explicación de la Arquitectura General

La arquitectura sigue el patrón Productor/Consumidor con el Monitor actuando como el intermediario. Los hilos(filósofos) actúan como consumidores/productores de los tenedores y el MonitorFilosofos actúa como el árbitro que controla las transiciones de estado de los filósofos, asegurando la exclusión mutua.

El diseño se basa en un array de estados para mantener la información de cada filósofo en un único lugar, lo que facilita la verificación del invariante de seguridad antes de permitir cualquier cambio de estado crítico.

Descripción de las clases

- **MonitorFilósofos** → Núcleo de Sincronización. Almacena el estado de los N filósofos. Garantiza la exclusión mutua sobre el estado mediante *synchronized*. Implementa la lógica de prevención de *deadlock*.
- **Filosofo** → Representa el Hilo. Implementa Runnable. Ciclo de vida: pensar() → tomarTenedores() → comer() → dejarTenedores(). Realiza las llamadas al Monitor para sincronizarse.
- **CenaFilosofos** → Configuración. Inicializa el Monitor y lanza los N hilos. (Thread) de los filósofos.

Decisiones de Diseño: Uso de monitores

Se eligió el Monitor de Hoare(implementado con *synchronized*, *wait()*, *notifyAll()* por su claridad y encapsulación de la lógica de sincronización.

- Justificación de la Solución: A diferencia de una solución basada puramente en Semáforos para cada tenedor, donde la lógica de la toma debe ser coordinada externamente, el Monitor centraliza la decisión de acceso. La única forma de entrar en la sección crítica(tomarTenedores o dejarTenedores) es adquiriendo el lock del Monitor.
- Prevención de DeadLock Inherente: La solución implementada verifica que ambos vecinos no estén comiendo antes de permitir que un filósofo entre en estado COMIENDO. Esta verificación se realiza de forma atómica dentro del método intentarComer(), que está protegido por el Monitor. Si la condición falla, el filósofo entra en espera condicional(*wait()*) sin tomar ningún recurso.

3. Implementación Concurrente

Detalle del Funcionamiento de MonitorFilosofos

Estados e Invariante

El Monitor usa tres estados para cada Filósofo:

- **PENSANDO** → Valor = 0. No necesita recursos
- **HAMBRIENTO** → Valor = 1. Necesita recursos, está en espera de la condición.
- **COMIENDO** → Valor = 2. Ha tomado los dos tenedores.

Uso de `synchronized` y `wait()`/`notifyAll()`

- **Synchronized:** Aplicado a `tomarDecisiones()` y `dejarTenedores()`. Garantiza que solo un hilo a la vez puede modificar el array `estado[]`
- **tomarTenedores(i):** El filósofo se declara HAMBRIENTO. Si la condición de seguridad (`intentarComer()`) no se cumple, el hilo entra en el bucle `while` (`estado[i] != COMIENDO`) y llama a `wait()`. El hilo pasa a estado `WAITING`, liberando el lock del monitor para que otro hilo pueda entrar y hacer progresar el sistema.
- **dejarTenedores(i):** El filósofo pasa a PENSANDO. Llama a `intentarComer()` para sus dos vecinos, dándoles la oportunidad inmediata de comer. Finalmente, `notifyAll()` se usa para despertar a todos los hilos que estaban en `WAITING` para que reevalúen su condición.

Justificación de cómo se evita el *deadlock* y cómo se reduce la *starvation*

DeadLock: Sincronización Condicional Atómica. El filósofo solo puede adquirir el estado `COMIENDO` si ambos recursos están disponibles (vecinos no comiendo). Si no, espera sin tomar ningún recurso, rompiendo el ciclo de espera circular.

Starvation: Uso de `HAMBRIENTO` y `notifyAll()`. Al despertar a todos (`notifyAll()`), se da la oportunidad a todos los hilos que esperan. El estado `HAMBRIENTO` garantiza que un vecino que ha estado esperando tiene prioridad para ser evaluado tan pronto como el tenedor se libera, lo cual reduce la posibilidad de inanición, aunque la equidad total depende del planificador de hilos de la JVM.

4. Pruebas y Resultados

Esta sección documenta la validación de la solución concurrente mediante pruebas unitarias y la observación del comportamiento del sistema.

Descripción de las Pruebas Realizadas

La validación se llevó a cabo mediante la ejecución de la aplicación principal (CenaFilosofos.java) y el uso de pruebas unitarias Junit 5. Se utilizó un escenario N = 5 filósofos para maximizar la posibilidad de interbloqueo(deadlock) y condiciones de carrera.

Las pruebas unitarias se agrupan en dos objetivos principales:

- Validación del Monitor
- Validación del Hilo

Resumen de las Pruebas Unitarias Implementadas con Junit

- **MonitorFilosofoTest.testExclusionMutuaVecinos()** → Se verifica que la Zona Crítica(mientras el filósofo está COMIENDO), ningún vecino tenga simultáneamente el estado COMIENDO. Si invarianteViolada es true, la prueba falla. Resultado: PASS. La condición "if(monitor.getEstado(izq) == COMIENDO.
- **MonitorFilosofosTest.testAusenciaDeDeadlock()** → Después de un tiempo fijo de simulación(4000ms), se verifica que el conteoComidas[i] sea mayor que 5 para todos los filósofos. Resultado: PASS. Se demostró que todos los hilos progresaron y comieron varias veces, confirmando la ausencia de deadlock.
- **FilosofoTest.testTerminacionPorInterrupcion()** → El hilo(Thread t) debe terminar su ejecución (!t.isAlive()) después de recibir una señal de interrupción (t.interrupt()). Resultado: PASS. Confirma que el bucle while (! Thread.currentThread().isInterrupted()) en el método run() del filósofo maneja correctamente la interrupción.

Selección de Capturas de Pantalla del Depurador

Captura A: Espera condicional y estado waiting

```

File Edit Source Refactor Navigate Search Project Run Copilot Window Help
D x P MonitorFilosofos Console x Problems Debug Shell
CenaFilosofos [Java Application] /home/usuario/.p2/pool/plugins/org.eclipse.justj...
Filósofo 0 PENSANDO...
Filósofo 1 PENSANDO...
Filósofo 2 PENSANDO...
Filósofo 3 PENSANDO...
Filósofo 4 PENSANDO...
Filósofo 0 tiene HAMBRE.
Filósofo 0 COMIENDO 🍽️
Filósofo 2 tiene HAMBRE.
Filósofo 2 COMIENDO 🍽️
Filósofo 1 tiene HAMBRE.
Filósofo 3 tiene HAMBRE.
Filósofo 4 tiene HAMBRE.
IntentarComer(1);
// Si no ha podido empezar a comer, espera hasta que
while (estado[i] != COMIENDO) {
    wait(); // pasa a estado WAITING asociado a este
}
}
/**
 * Filósofo i deja de comer y pasa a pensar. Después de l
 * tenedores, permite que sus vecinos intenten comer.
 *
 * @param i índice del filósofo (0..N-1)
 */

```

La captura muestra el hilo Filósofo-X detenido en la línea `wait()` del método `tomarTenedores()`. Su estado en el Monitor es `HAMBRIENTO(1)`, pero la condición de seguridad no se cumple porque uno de sus vecinos está en estado `COMIENDO(2)`. El Hilo pasa a `WAITING` y libera el lock, lo que es crucial para evitar el deadlock.

Captura B: Verificación de la Invariante

```

83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120

// Sus vecinos pueden intentar comer ahora que i ha d
intentarComer(izquierda(i));
intentarComer(derecha(i));
notifyAll();
}

/**
 * Solo para depuración o para mostrar el estado en una i
 *
 * @param i índice del filósofo
 * @return estado del filósofo i
 */
public synchronized int getEstado(final int i) {
    return estado[i];
}

public int getLeftIndex(final int i) {
    return izquierda(i);
}

public int getRightIndex(final int i) { return derecha(i)

```

La imagen ilustra la verificación de la Invariante de Seguridad. El array de estado (`estado[ ]`) muestra un patrón donde no hay dos filósofos vecinos en estado `COMIENDO`(2). Esta atomicidad garantiza la exclusión mutua sobre los tenedores, demostrando que la lógica de `intentarComer()` funciona correctamente antes de que el filósofo `F`, entre en la zona crítica.

Captura C: Notificación de Progreso

```

48     }
49
50     priv
51     Filosofo 0 PENSANDO...
52     Filosofo 1 PENSANDO...
53     Filosofo 2 PENSANDO...
54     Filosofo 3 PENSANDO...
55     priv
56     Filosofo 4 PENSANDO...
57     Filosofo 3 tiene HAMBRE.
58     Filosofo 2 tiene HAMBRE.
59     Filosofo 0 tiene HAMBRE.
60     Filosofo 1 tiene HAMBRE.
61     Filosofo 4 tiene HAMBRE.
62
63     /**
64     * I
65     * n
66     * @
67     */
68     priv
69     if (estado[i] == HAMBRIENTO
70         && estadoIzq(i) != COMIENDO
71         && estadoDer(i) != COMIENDO) {
72         cambiarEstado(i, COMIENDO);
73         // Despertamos a todos para que los filósofos que
74         // esperando puedan reevaluar su condición.
75         notifyAll();
76     }
77
78     /**
79     * Filósofo i se declara hambriento e intenta comer. Si n
80     * el hilo queda bloqueado (WAITING) hasta que pueda pasa
81     * @param i índice del filósofo (0..N-1)
82     * @throws InterruptedException si el hilo es interrumpid
83     */
84     public synchronized void tomarTenedores(final int i) thro
85     cambiarEstado(i, HAMBRIENTO);
86     intentarComer(i);
87

```

La captura se tomó en la línea `notifyAll()` del método `dejarTenedores()`. El hilo actual ha pasado a `PENSANDO(0)`. La llamada despierta a todos los hilos que estaban en estado `WAITING` (como se muestra en la foto), permitiéndoles reevaluar la condición de comida y, por ende, asegurando la vivacidad y el progreso del sistema.

Comentarios sobre el comportamiento observado

El diseño basado en el monitor centralizado demostró ser seguro y robusto. La solución logra una buena equidad en la distribución de turnos, aunque la solución con `notifyAll()` puede ser menos eficiente que el uso de condiciones específicas (`java.util.concurrent.locks.Condition`). Se confirma que el interbloqueo está ausente, ya que los hilos no retienen recursos mientras esperan la condición de comer.

5. Resumen de lo aprendido

La práctica ha consolidado la comprensión de la sincronización condicional en Java, destacando la diferencia entre la exclusión mutua (*synchronized*) y la espera de condición (*wait()/notifyAll()*). Se logró modelar y resolver un problema clásico de la concurrencia, demostrando un diseño que es seguro y libre de interbloqueo.

6. Enlace repositorio en GIT

<https://github.com/ananutlo/CenaFilosofos>